R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Gestion des processus sous Unix/Linux, multitâche préemptif, cycle de vie (Élu, Prêt, Bloqué), arborescence (PID/PPID) et signaux POSIX

Synthèse exécutive : Ce cours aborde les fondements essentiels de l'architecture des systèmes d'exploitation Unix/Linux en expliquant la transition entre un programme binaire statique stocké sur disque et un processus actif s'exécutant en mémoire RAM. Afin de permettre l'exécution simultanée de multiples applications sur une même machine, le noyau (kernel) s'appuie sur le multitâche préemptif et gère le cycle de vie des processus via trois états principaux : Élu, Prêt et Bloqué. La séance détaille la filiation arborescente des processus (PID et PPID) issue du processus initial systemd/init (PID 1), le rôle des appels système fork() et exec(), ainsi que le fonctionnement du mécanisme de communication inter-processus asynchrone : les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour exécuter un code informatique, le système d'exploitation doit faire basculer le programme (ensemble d'instructions statiques stockées sur le stockage secondaire) vers l'état de processus (entité active chargée en mémoire vive RAM). Le noyau Linux orchestre ce fonctionnement grâce au multitâche préemptif : le temps du processeur est découpé en minuscules quanta de temps attribués successivement aux processus éligibles. L'ordonnanceur (scheduler) gère le passage dynamique des processus entre trois états fondamentaux : 1. Élu (Running) : Le processus est pris en charge par un cœur du processeur et exécute ses instructions machine. 2. Prêt (Ready) : Le processus dispose de toutes les ressources mémoire nécessaires mais attend que l'ordonnanceur lui attribue son prochain quantum de temps CPU. 3. Bloqué (Waiting / Blocked) : Le processus a déclenché une requête nécessitant une attente externe (ex: lecture de fichier sur disque, attente d'un paquet réseau ou saisie utilisateur au clavier). Pour éviter de pénaliser les performances globales, le noyau libère le CPU et place le processus en attente d'événement. Dès que l'événement survient, le processus repasse obligatoirement par l'état 'Prêt' avant de pouvoir être réélu. Tous les processus au sein du système d'exploitation s'inscrivent dans une arborescence généalogique stricte. Lors de l'initialisation du système, le noyau crée le tout premier processus utilisateur, nommé systemd (ou init sur les distributions plus anciennes), auquel est assigné le PID 1. Lors de la création d'un sous-processus (par exemple lorsqu'un utilisateur lance une commande dans un terminal Bash), le terminal (processus parent) appelle la fonction système fork() pour se cloner. Le processus fils ainsi généré possède un nouveau PID et hérite du PID du parent (PPID). L'enfant exécute ensuite l'appel système exec() pour substituer sa carte mémoire par le code binaire du programme souhaité.

- • Programme vs Processus : Un programme est un fichier binaire passif stocké sur disque dur (ex: /usr/bin/python3). Un processus est son instance dynamique en cours d'exécution en mémoire RAM, consommant des cycles CPU et disposant d'attributs propres.
- • Multitâche préemptif : Fonctionnement par lequel l'ordonnanceur (scheduler) du noyau alloue de très courtes tranches de temps processeur (quanta) aux processus pour simuler l'exécution simultanée de multiples tâches.
- • Bloc de Contrôle du Processus (PCB) : Structure de données interne au noyau qui conserve l'état complet du processeur, la carte mémoire (stack, heap, code, data), les identifiants (PID, PPID, UID), la priorité et les descripteurs de fichiers ouverts.
- • Filiation PID / PPID & Processus Racine : Chaque processus possède un identifiant unique (PID). Il dérive d'un processus père (PPID). L'arbre généalogique remonte jusqu'au tout premier processus utilisateur, systemd ou init (PID 1).
- • Appels système fork() et exec() : fork() duplique le processus parent à l'identique (créant un processus enfant avec son propre PID), puis exec() remplace le code exécutable du processus enfant par celui du nouveau programme à exécuter.

• Cycle de Vie à 3 États : Élu (Running - occupe le CPU), Prêt (Ready - possède ses ressources RAM et attend du temps CPU), Bloqué (Waiting/Blocked - en attente d'une opération d'E/S disque, réseau ou frappe clavier). La transition se fait de Bloqué vers Prêt, jamais directement de Bloqué vers Élu.

Cas concret enseignant : *L'analogie du livre de recettes : Le programme sur disque dur est comparé à un livre de recettes rangé sagement dans une bibliothèque (statique, passif, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier qui prépare activement la recette dans sa cuisine (dynamique en RAM, occupant du plan de travail, consommant de l'énergie CPU et manipulant des ustensiles/descripteurs de fichiers).*

Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme de communication inter-processus (IPC - Inter-Process Communication) la plus rudimentaire et universelle des systèmes Unix. Un signal est une notification asynchrone : lorsqu'il est délivré à un processus cible, l'exécution courante de ce dernier est interrompue instantanément pour traiter l'événement. À la réception d'un signal, trois options s'offrent au processus selon sa programmation : 1. Exécuter l'action par défaut définie par le système (généralement la terminaison du processus ou l'ignorance du signal). 2. Ignorer le signal (ignore). 3. Intercepter le signal (catch) en déroutant le fil d'exécution vers une fonction dédiée appelée 'gestionnaire de signal' (signal handler). Cela permet par exemple à un logiciel de traitement de texte d'intercepter SIGINT ou SIGTERM pour afficher le message : 'Voulez-vous sauvegarder vos modifications avant de quitter ?'. Cependant, pour conserver un contrôle absolu sur le système et permettre à l'administrateur de neutraliser n'importe quel processus défaillant ou malveillant, la norme POSIX définit deux exceptions majeures : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces deux signaux sont interceptés et exécutés directement par le noyau lui-même. Aucun programme utilisateur ne peut définir de gestionnaire pour SIGKILL ou SIGSTOP, les rendant inévitables.

• Signal POSIX : Interruption logicielle asynchrone transmise à un processus pour lui notifier un événement système ou lui ordonner une action spécifique.

• Gestionnaire de signal (Signal Handler) : Fonction personnalisée qu'un développeur peut attacher à un signal pour intercepter (catch) le signal au lieu d'exécuter l'action par défaut du système.

• SIGINT (Signal 2) : Émis lors du raccourci Ctrl + C dans le terminal. Demande une interruption douce et modérée d'une application interactive.

• SIGTERM (Signal 15) : Signal d'arrêt propre envoyé par défaut par la commande 'kill '. Il autorise le processus à exécuter des procédures de nettoyage (sauvegarde de fichiers, fermeture de sockets réseau, suppression de fichiers temporaires) avant de s'éteindre.

• SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel géré directement par le noyau Linux. Il détruit immédiatement le processus. Il est STRICTEMENT IMPOSSIBLE à intercepter (catch), masquer ou ignorer par le programme.

• Job Control (SIGSTOP 19 / SIGCONT 18) : Ctrl + Z envoie SIGSTOP pour suspendre l'exécution d'un processus. Les commandes 'fg' (foreground) ou 'bg' (background) envoient SIGCONT pour reprendre l'exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous avez le temps de sauvegarder vos documents et fermer la porte à clé). SIGKILL (Signal 9) est une intervention d'urgence de la brigade de sécurité qui défonce la porte et vous dégage sur-le-champ sans vous laisser toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : En syntaxe shell Bash, la commande kill attend comme option le numéro de signal précédé d'un tiret (-9 ou -SIGKILL) suivi du PID de la cible (1234). Si vous écrivez 'kill 9 1234' sans tiret, le shell interprète 9 et 1234 comme deux PIDs distincts et tente d'envoyer le signal par défaut (SIGTERM - 15) aux processus ayant les PIDs 9 et 1234. Le tiret est donc syntaxiquement obligatoire pour indiquer au shell qu'il s'agit du numéro du signal et non d'un PID.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie (marqué d'un état Z dans la commande ps) est un processus enfant qui a terminé son exécution, mais dont le père n'a pas encore lu le code de retour de fin (via l'appel système wait() ou waitpid()). Le processus zombie n'occupe plus aucune mémoire RAM ni temps CPU, mais sa structure persiste dans la table des processus du noyau. Il est impossible de tuer un zombie avec 'kill -9 ' car le processus est DEJÀ mort ! Pour faire disparaître un zombie, il faut soit inciter le processus père à exécuter wait(), soit tuer le processus père ('kill -9 ') : l'enfant zombie sera alors réadopté par le processus racine systemd/init (PID 1) qui procèdera immédiatement au nettoyage de la table des processus.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez ce qui se passe lors de l'envoi du signal -15 et décrivez le résultat de la vérification ultérieure par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche l'identifiant du job (ex: [1]) ainsi que le PID attribué par le noyau (ex: 4521) et rend immédiatement la main à l'utilisateur pour qu'il puisse continuer à saisir d'autres commandes dans la session Bash. Étape 2 : La commande 'ps -ef | grep sleep' filtre la table de tous les processus système actifs et affiche la ligne correspondant au processus 'sleep 600'. Cela permet de repérer son PID (4521) ainsi que son PPID (qui correspond au PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. Comme l'utilitaire 'sleep' ne dispose pas d'un gestionnaire personnalisé interceptant SIGTERM, l'action par défaut associée s'applique : l'arrêt propre du processus.

Résultat : Le processus 'sleep 600' est immédiatement terminé de façon propre. Une nouvelle exécution de la commande 'ps -ef | grep sleep' confirme sa disparition définitive de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le premier processus utilisateur est systemd (ou init sur les systèmes plus anciens), et son PID est toujours 1.

Q : Quel signal POSIX est généré et transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison de touches 'Ctrl + C' ?

R : C'est le signal SIGINT (Signal 2 - Signal Interruption).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté, masqué ou ignoré par un programme informatique ?

R : Parce que le signal SIGKILL est intercepté et exécuté directement par le noyau (kernel) du système d'exploitation, sans jamais transmettre le contrôle au gestionnaire de signal du processus utilisateur. Cela garantit à l'administrateur système un moyen infaillible de stopper un processus.